

ICT 6127

Advanced TCP/IP Inter-networking

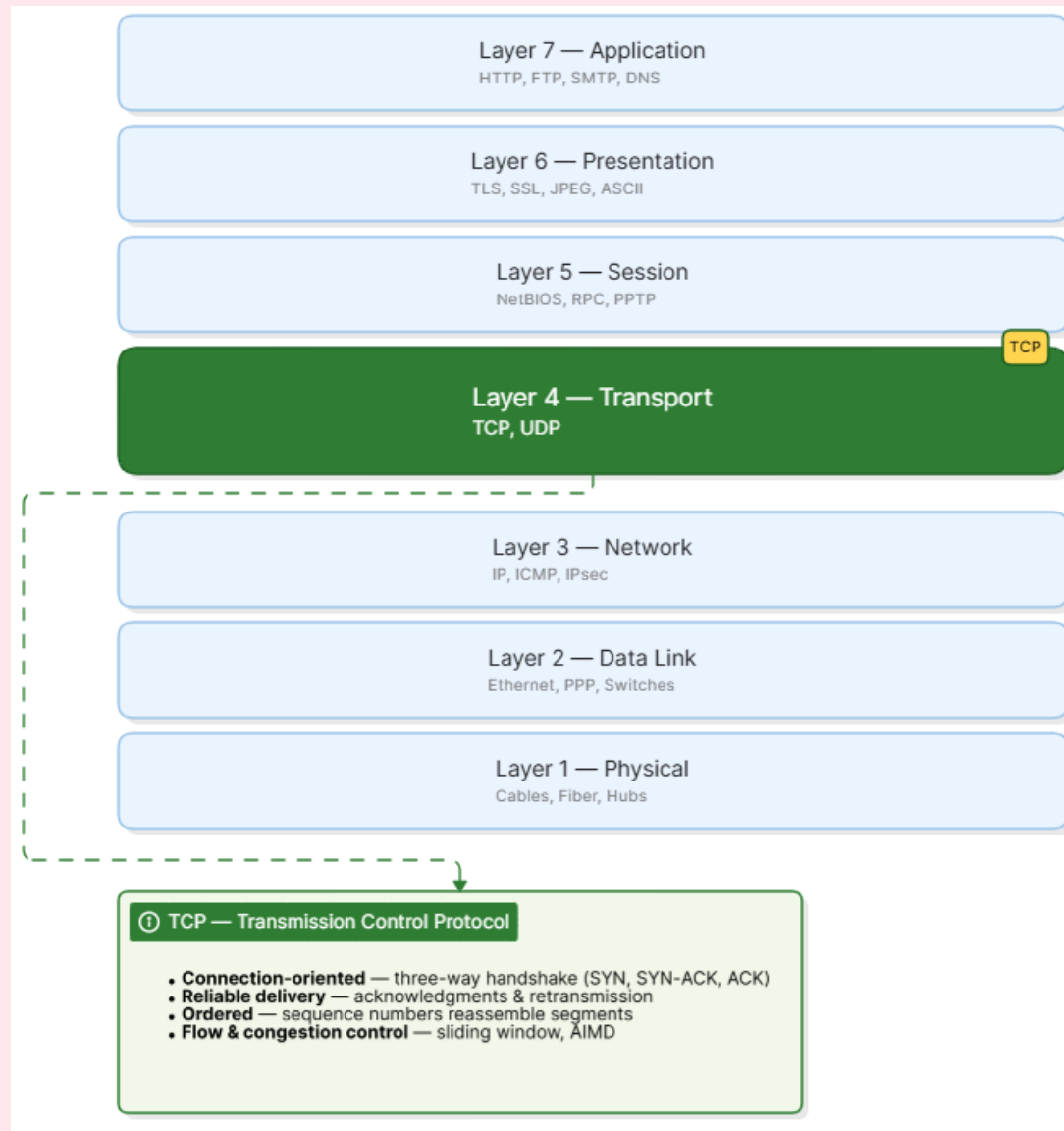
Transmission Control Protocol (TCP)

Presented to **DR NAZRUL ISLAM**

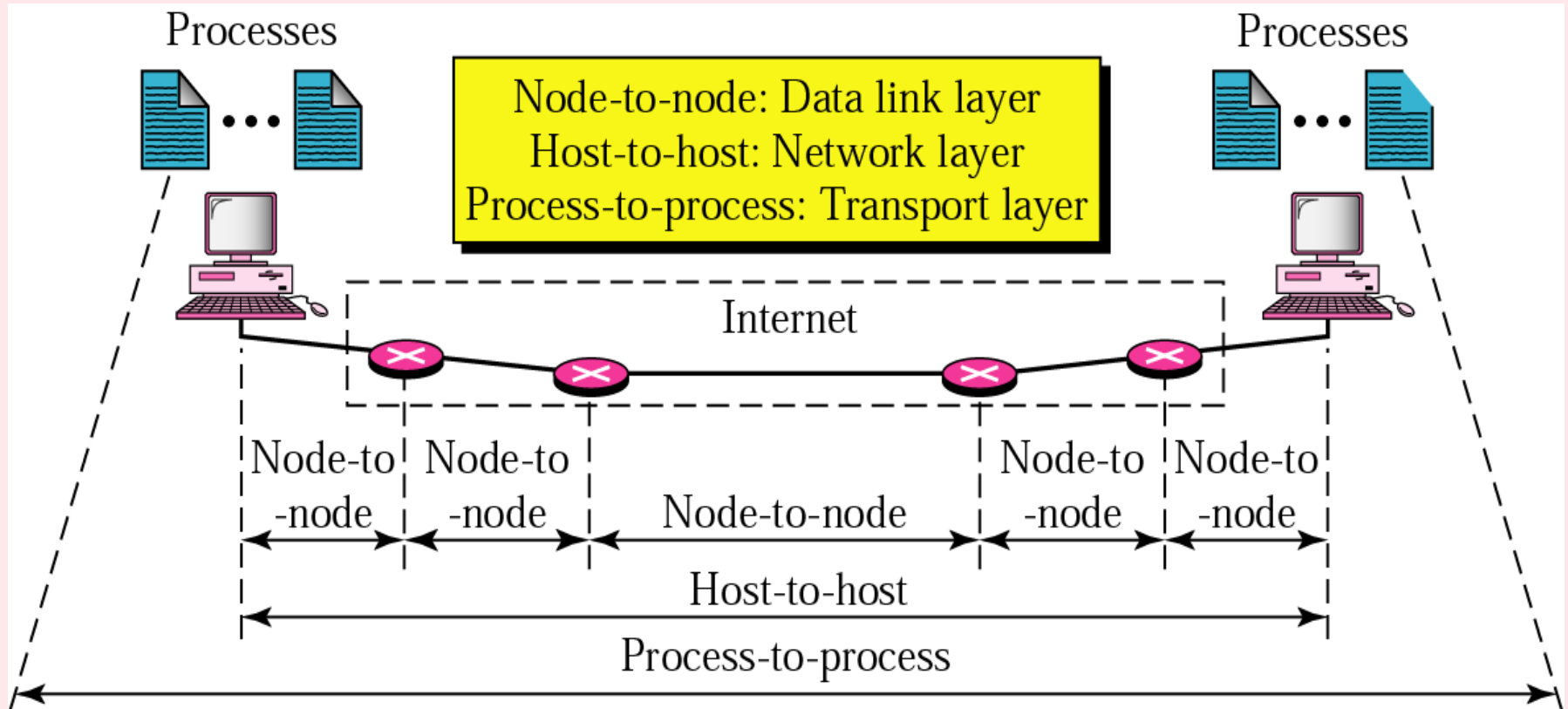
Department of ICT, MBSTU

by

R Tamanna, IT25641



Transport Layer Contd.

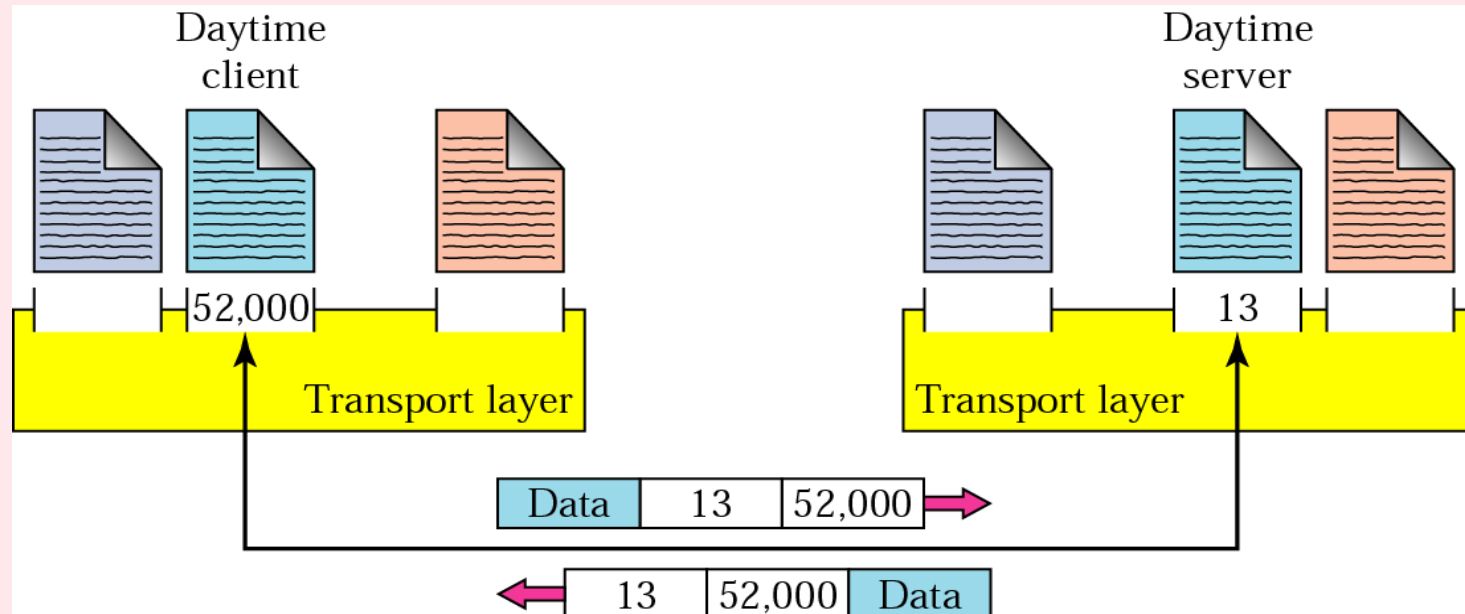


Process-to-process delivery

Transport Layer Addressing

Addresses

- Data link layer → MAC address
- Network layer → IP address
- Transport layer → *Port number* (choose among multiple processes running on destination host)



Port Numbers

- Port numbers are 16-bit integers (0 → 65,535)
 - Servers use *well know ports*, 0-1023 are privileged
 - Clients use *ephemeral* (short-lived) ports
- *Internet Assigned Numbers Authority* (IANA) maintains a list of port number assignment
 - **Well-known ports** (0-1023) → controlled and assigned by IANA
 - **Registered ports** (1024-49151) → IANA registers and lists use of ports as a convenience (49151 is $\frac{3}{4}$ of 65536)
 - **Dynamic ports** (49152-65535) → ephemeral ports
- For well-known port numbers, see /etc/services on a UNIX or Linux machine

Socket Addressing

- Process-to-process delivery needs *two* identifiers
 - IP address and Port number
 - Combination of IP address and port number is called a socket address (a socket is a communication endpoint)
 - Client socket address uniquely identifies client process
 - Server socket address uniquely identifies server process
- Transport-layer protocol needs a *pair* of socket addresses
 - Client socket address
 - Server socket address
 - For example, socket pair for a TCP connection is a 4-tuple
 - ✓ Local IP address, local port, and
 - ✓ foreign IP address, foreign port

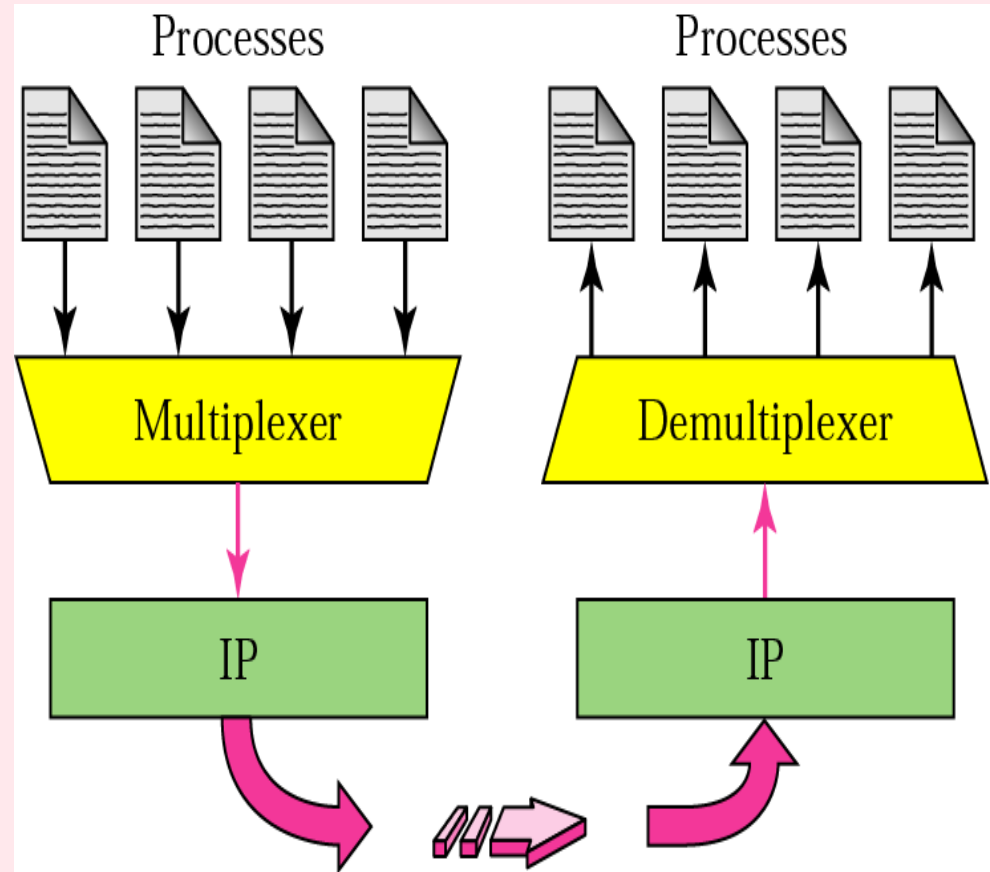
Multiplexing and Demultiplexing

Multiplexing

Sender side may have several processes that need to send packets (albeit only 1 transport-layer protocol)

Demultiplexing

At receiver side, after error checking and header dropping, transport-layer delivers each message to appropriate process



Transmission Control Protocol 2/10

•Reliable

- requires ACK and performs retransmission
- If ACK not received, retransmit and wait a longer time for ACK. After a number of retransmissions, will give up
- How long to wait for ACK? (dynamically compute RTT for estimating how long to wait for ACKs, **might be ms for LANs or seconds for WANs**)

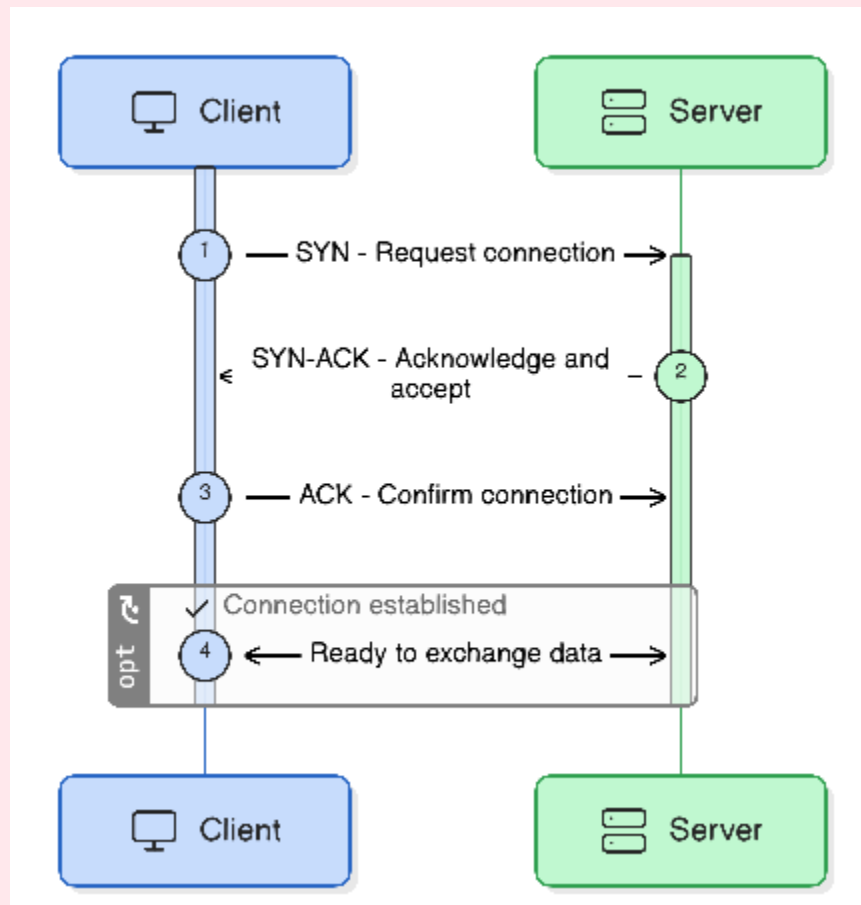
$RTT = \alpha * \text{old RTT} + (1 - \alpha) * \text{new RTT}$ where α usually 90%

- Most common, Retransmission time = 2* RTT
- Acknowledgments can be “piggy-backed” on reverse direction data packets or sent as separate packets

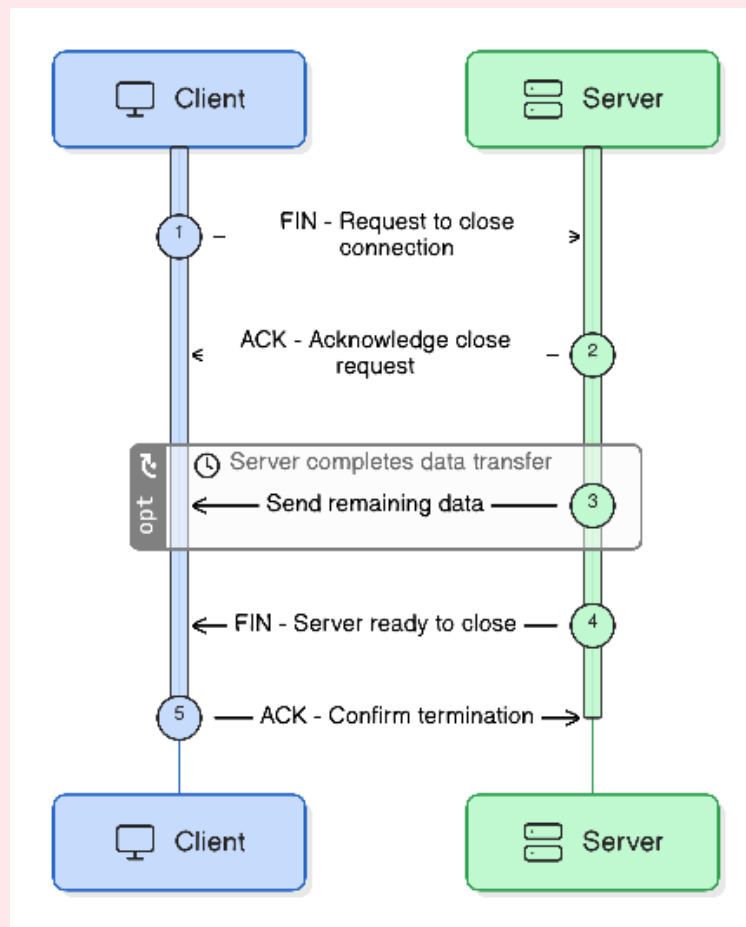
Transmission Control Protocol ^{3/10}

•Sequence Numbers

- Associated with every byte that it sends
- To detect packet loss, reordering and duplicate removal
- Two fields are used *sequence number* and *acknowledgment number*. Both refer to byte number and not segment number
- Sequence number for each segment is the number of the **first byte** carried in that segment
- The ACK number denotes the number of the **next byte** that this party expects to receive (cumulative)
 - ✓ If an ACK number is 5643 → received all bytes from beginning up to 5642
 - ✓ This acknowledges all previous bytes as received error-free



- **SYN (Synchronize):** The sender sends a SYN segment to the receiver to request a connection.
- **SYN-ACK (Synchronize-Acknowledge):** The receiver responds with a SYN-ACK segment, acknowledging the request and agreeing to the connection.
- **ACK (Acknowledge):** The sender replies with an ACK, confirming the connection is established



Closing a TCP connection requires a four-step handshake to ensure both sides finish transmitting data safely:

FIN (Finish): The sender who wants to close the connection sends a FIN segment to the receiver.

ACK (Acknowledge): The receiver acknowledges the FIN with an ACK.

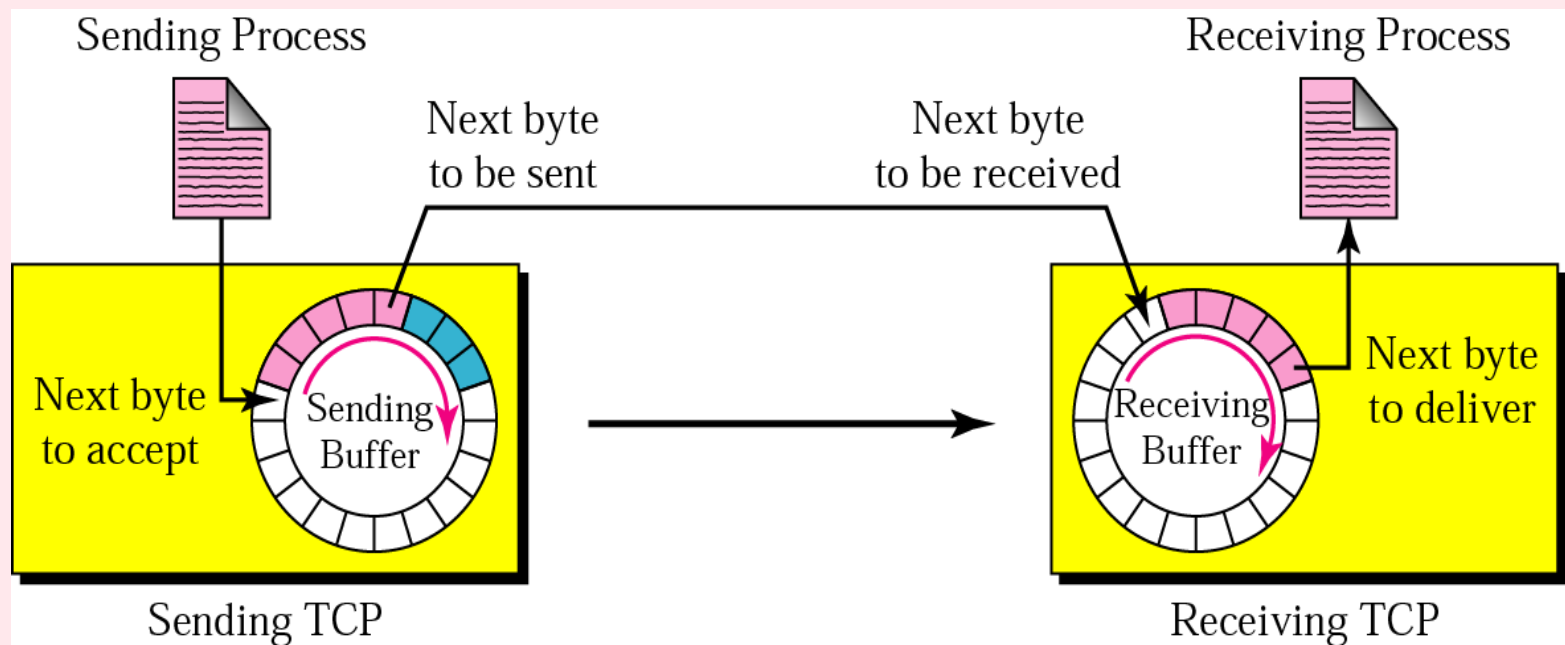
FIN (Finish) from Receiver: The receiver then sends its own FIN when it is ready to close the connection.

ACK (Acknowledge): The sender responds with an ACK, completing the termination.

Transmission Control Protocol ^{4/10}

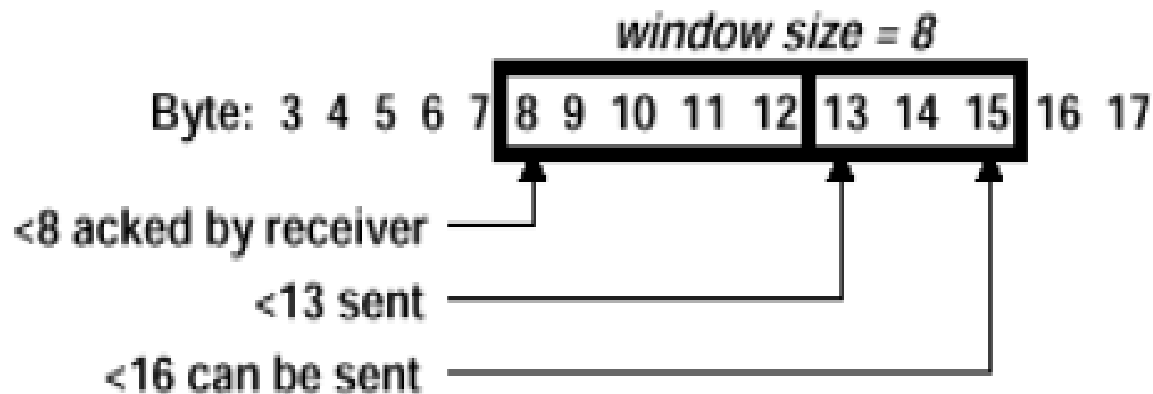
•Sending and Receiving buffers

- Senders and receivers may not produce and consume data at same speed
- 2 buffers for each direction (sending and receiving buffer)

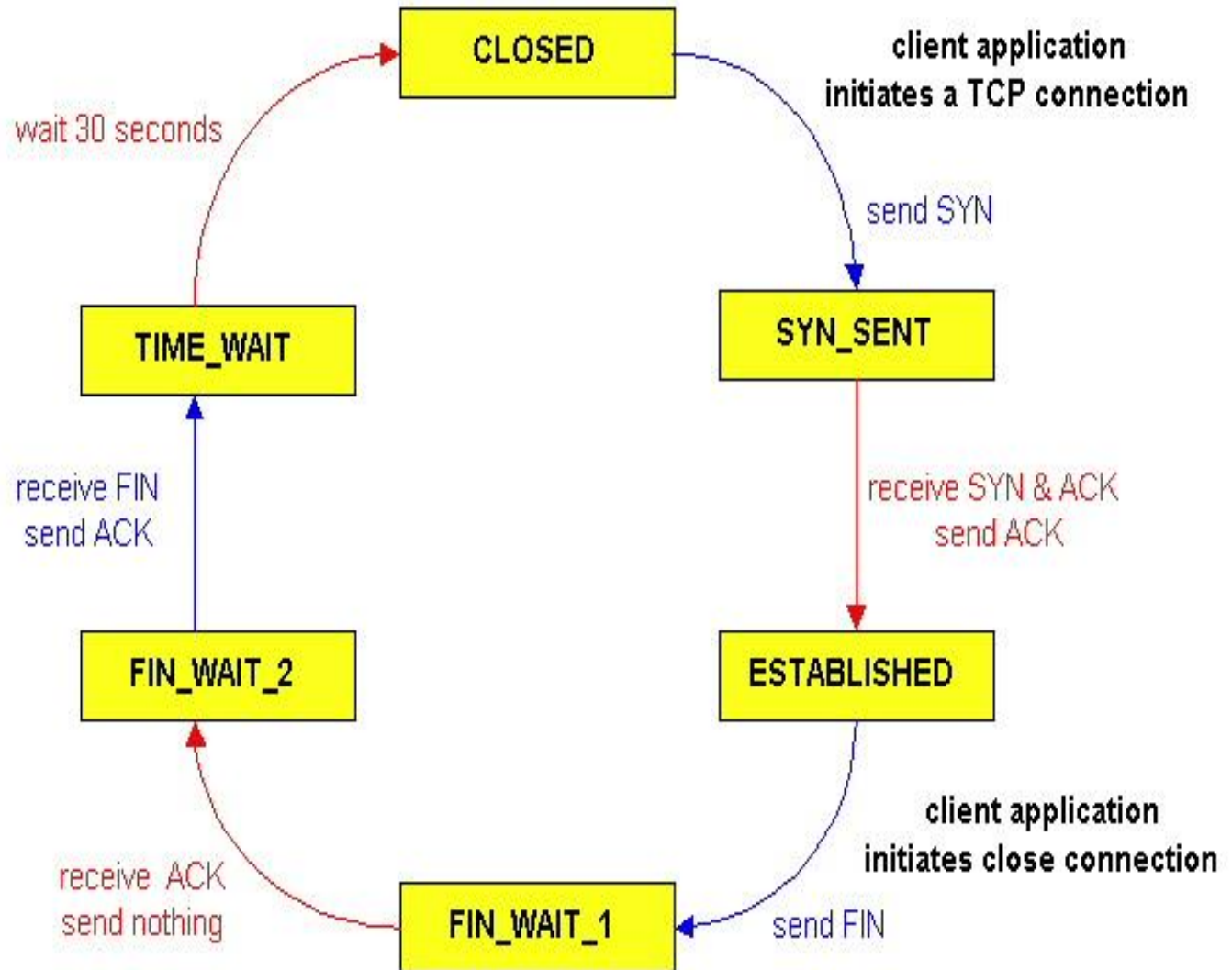


Transmission Control Protocol 5/10

- TCP uses a sliding window mechanism for flow control
- Sender maintains 3 pointers for each connection
 - Pointer to bytes sent and acknowledged
 - Pointer to bytes sent, but not yet acknowledged
 - ✓ *Sender window includes bytes sent but not acknowledged*
 - Pointer to bytes that cannot yet be sent



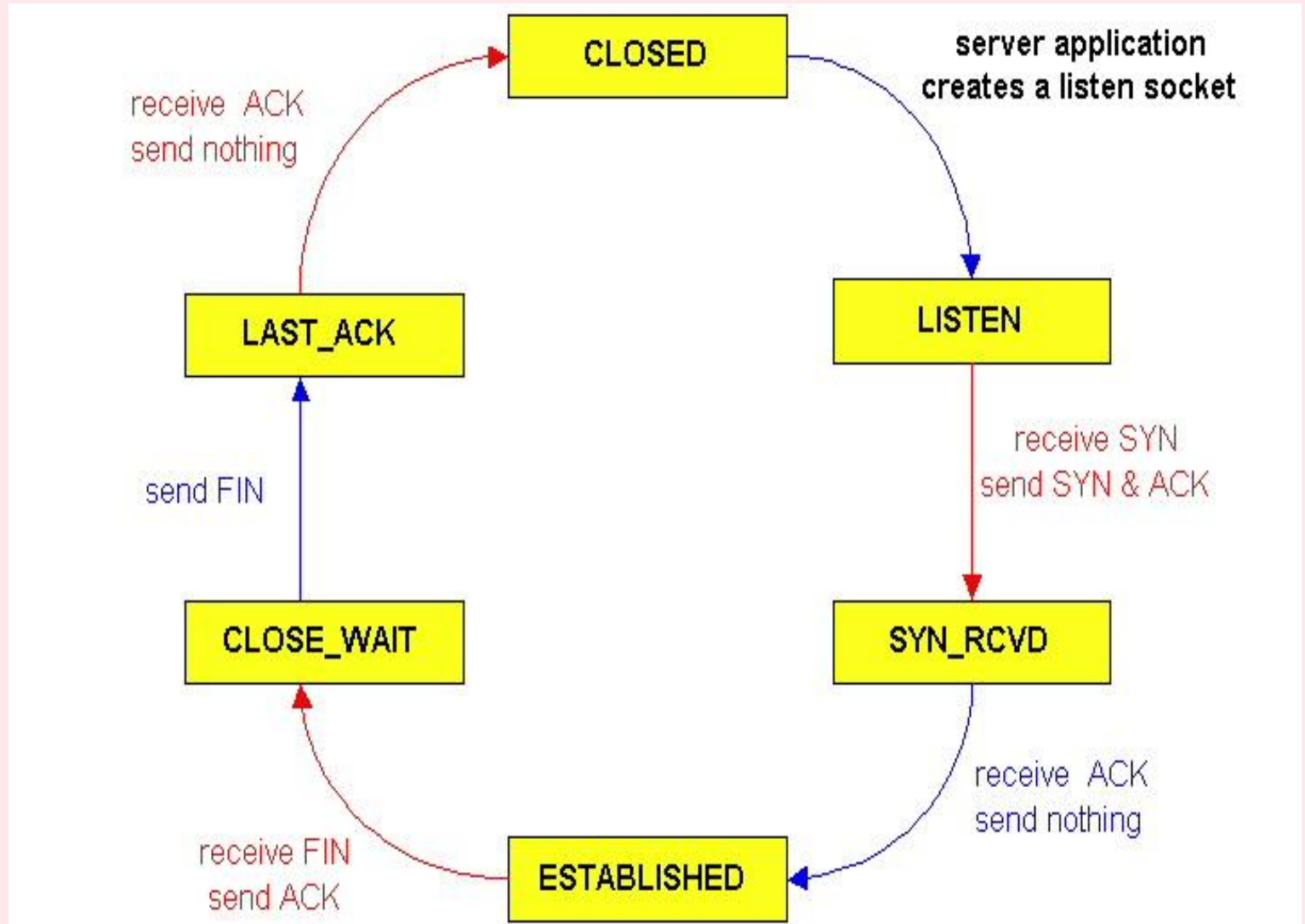
State Transition Diagram ^{1/4}



Typical TCP
states visited
by a TCP
client

State Transition Diagram 2/4

Typical
TCP
states
visited by
a TCP
server

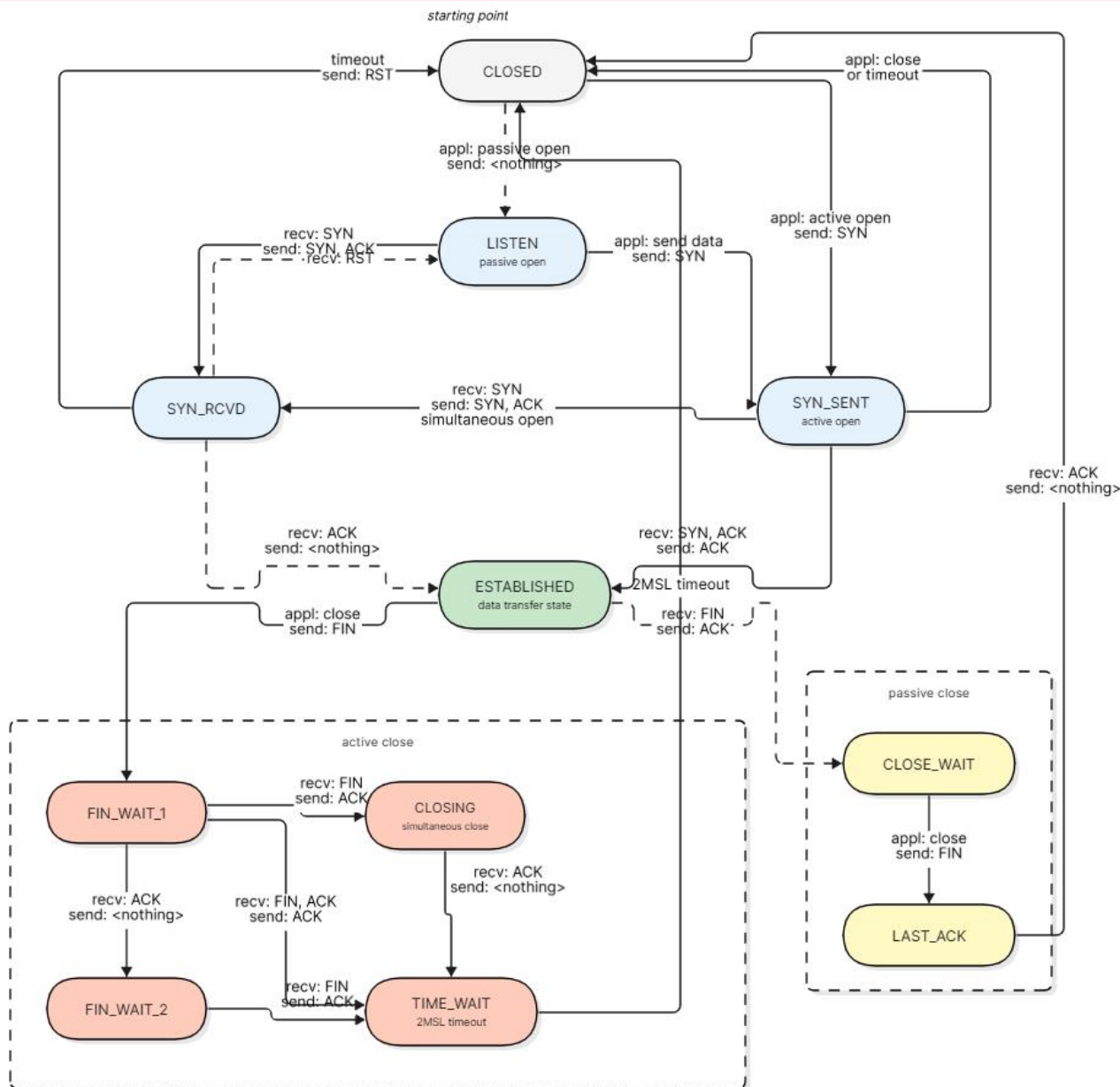


State Transition Diagram ^{3/4}

State	Description
CLOSED	There is no connection.
LISTEN	The server is waiting for calls from the client.
SYN-SENT	A connection request is sent; waiting for acknowledgment.
SYN-RCVD	A connection request is received.
ESTABLISHED	Connection is established.
FIN-WAIT-1	The application has requested the closing of the connection.
FIN-WAIT-2	The other side has accepted the closing of the connection.
TIME-WAIT	Waiting for retransmitted segments to die.
CLOSE-WAIT	The server is waiting for the application to close.
LAST-ACK	The server is waiting for the last acknowledgment.

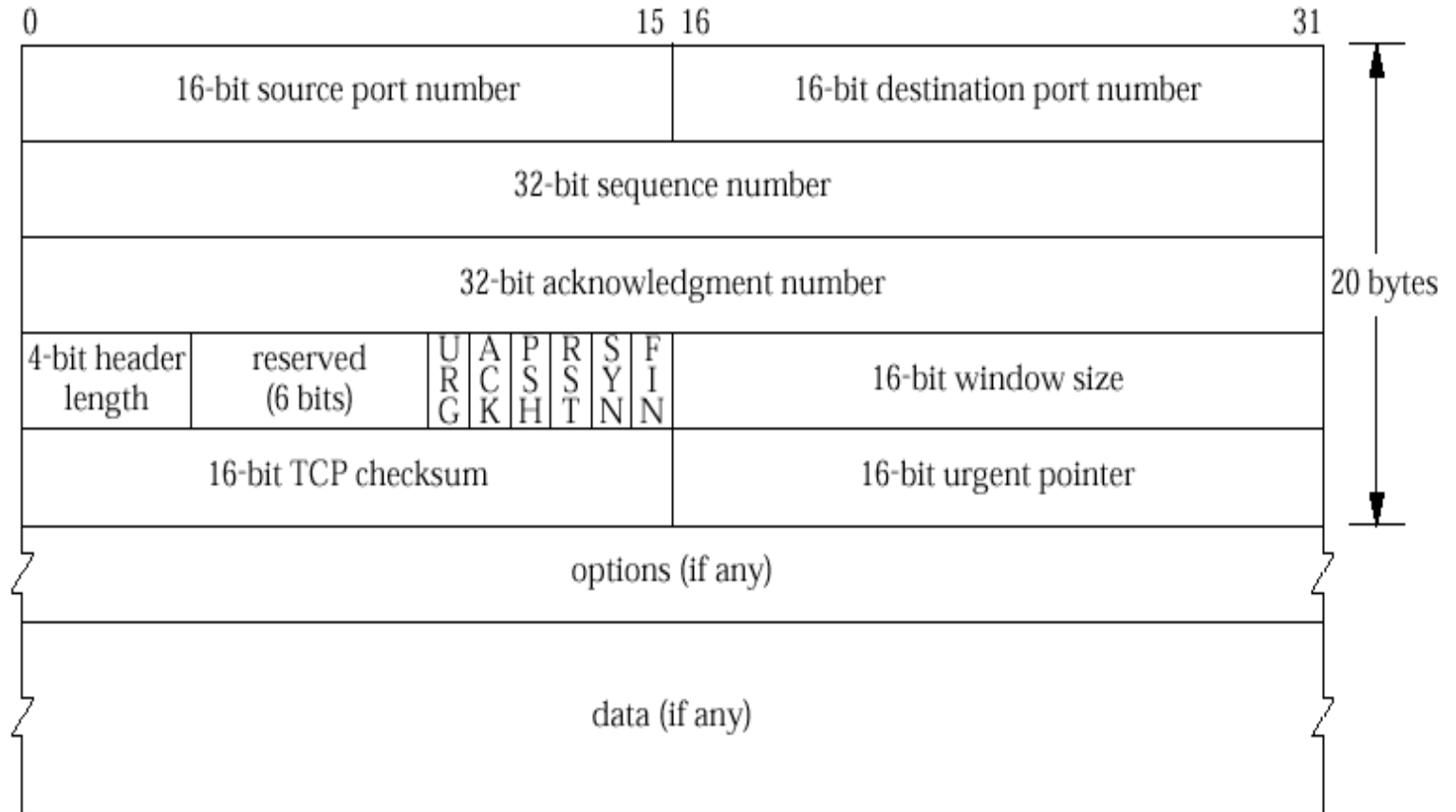
Can use *netstat* command to see some TCP states

State Transition Diagram 4/4



TCP Segment Format

TCP Header



TCP Header Fields ^{1/2}

- Source Port and Destination Port

- Identify processes at ends of the connection

- Control bits

- URG urgent (urgent data present)

- ACK acknowledgment

- PSH push request

- ✓Inform receiver TCP to send data to application ASAP

- RST reset the connection

- SYN synchronize sequence numbers

- FIN sender at end of byte stream

TCP Header Fields ^{2/2}

- **Sequence Number**: position of the data in the sender's byte stream
- **Acknowledgment Number**: position of the byte that the source expects to receive next (valid if ACK bit set)
- **Header Length**: header size in 32-bit units. Value ranges from [5-15]
- **Window**: advertised window size in bytes
- **Urgent**
 - ✓ defines end of urgent data (or “out-of-band”) data and start of normal data
 - ✓ Added to sequence number (valid only if URG bit is set)
- **Checksum**: 16-bit CRC (Cyclic Redundancy Check) over header and data
- **Options**: up to 40 bytes of options

References

- [1] R. Tamanna, *Personal Portfolio*. [Online]. Available: <https://rubayamist.github.io/>
- [2] R. Tamanna, *TCP/IP Internet Project*, GitHub Repository. [Online]. Available: <https://github.com/rubayamist/TCP-IPInterNet>
- [3] Virginia Tech, *TCP Protocol Lecture Slides (CS4254)*. [Online]. Available: <https://courses.cs.vt.edu/cs4254/spring06/slides/TCP.ppt>